JAVA INTERVIEW PREPARATION

CHAPTER 24

EXECUTORS AND THREAD POOLS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Executors, Thread Pools, Queues, and Task Lifecycle: Designing Bounded Execution

Senior / Lead Java Developer Reading Chapter

An Executor separates submission of work from the mechanism that runs it. That separation enables reuse, scheduling, concurrency limits, cancellation, monitoring, and shutdown. It also hides dangerous defaults: an unbounded queue can retain unlimited work, a fixed pool can deadlock when tasks wait for tasks in the same pool, and an ignored Future can hide every task failure.

Tasks Are Not Threads

A Runnable or Callable describes work. An Executor decides when and where the work runs.

```
Callable<Invoice> task = () -> generateInvoice(orderId);
Future<Invoice> future = executor.submit(task);
Invoice invoice = future.get();
```

Callable returns a result and may throw checked exceptions. Future represents eventual completion, failure, or cancellation. Submission does not imply that execution has started; a task may be queued or rejected.

ThreadPoolExecutor as a Capacity System

ThreadPoolExecutor is defined by interacting policies:

```
incoming task
     |
     v
fewer than core threads? ---- yes ---> create worker
     |
     no
     v
queue accepts task? ---------- yes ---> wait in queue
     |
     no
     v
fewer than maximum threads? -- yes ---> create extra worker
     |
     no
     v
rejection policy
```

The main parameters are core pool size, maximum pool size, keep-alive time, work queue, thread factory, and rejection handler. They must be designed together. With an unbounded queue, maximumPoolSize is normally irrelevant because tasks continue to queue after core threads are busy.

Queue Choice Changes Pool Behavior

An ArrayBlockingQueue or bounded LinkedBlockingQueue caps retained backlog. A SynchronousQueue has no storage and hands a task directly to a worker, encouraging thread growth up to the maximum. PriorityBlockingQueue orders tasks but is unbounded unless another control is added.

```
ThreadPoolExecutor executor = new ThreadPoolExecutor(
    16,
    32,
    30, TimeUnit.SECONDS,
    new ArrayBlockingQueue<>(500),
    namedThreadFactory("pricing-"),
    new ThreadPoolExecutor.CallerRunsPolicy()
);
```

This is an example, not a universal configuration. Capacity comes from measured service time, arrival rate, CPU, downstream limits, and tolerated queueing latency.

Queue Length Is Latency Stored in Memory

If work arrives faster than it completes, backlog grows.

```
arrival 1,000 tasks/s
completion 800 tasks/s
backlog grows 200 tasks/s
```

Each queued task retains arguments, request context, captured closures, and referenced object graphs. A queue of 100,000 tasks can represent minutes of stale latency and gigabytes of retained memory. Bounding the queue forces an explicit overload decision before the process fails unpredictably.

Little's Law connects average concurrency, throughput, and latency:

```
in-flight work approximately = throughput x average time in system
```

Use it to challenge impossible capacity assumptions, then validate with load tests and percentile latency.

Pool Sizing

CPU-bound work normally benefits from runnable concurrency near available cores, adjusted for measurement and other process activity. Excess threads create scheduling overhead and cache disruption.

Waiting workloads can use more threads because many are blocked, but the limiting dependency still determines safe concurrency. A common estimation model is:

```
threads approximately = cores x target utilization x (1 + wait time / compute time)
```

This is an initial hypothesis, not a production truth. Measure real wait ratios, downstream capacity, heap cost per task, and latency. Virtual threads change the cost of waiting threads but not resource capacity; use explicit permits for scarce dependencies.

Rejection Is Part of the API Contract

When both workers and queue are saturated, the rejection handler runs.

- AbortPolicy throws RejectedExecutionException.
- CallerRunsPolicy executes in the submitting thread and can slow producers.
- DiscardPolicy silently drops work.
- DiscardOldestPolicy removes queued work and retries submission.

Silent discard is rarely safe for business work. CallerRuns can provide feedback in some internal pipelines, but on an event-loop or request thread it may violate latency or threading assumptions. Map saturation to an intentional response: backpressure, HTTP 429 or 503, message retry, load shedding, or durable overflow.

execute Versus submit and Lost Exceptions

With execute, an uncaught RuntimeException reaches the worker thread's uncaught-exception handling and may replace that worker. With submit, failure is captured by the Future and appears from get as ExecutionException.

```
Future<?> future = executor.submit(() -> {
    throw new IllegalStateException("failed");
});

future.get(); // throws ExecutionException with the original cause
```

Submitting and discarding the Future can make failure invisible. Establish a policy: observe futures, decorate tasks, override afterExecute carefully, or use framework-level error reporting. Metrics should count completed, failed, rejected, and cancelled tasks separately.

Cancellation and Interruption

`future.cancel(true)` requests interruption if the task is running. It does not forcibly stop arbitrary code.

```
while (!Thread.currentThread().isInterrupted()) {
    performUnit();
}
```

Interruptible blocking calls cooperate automatically by throwing InterruptedException. Libraries and loops must preserve the signal. A task performing non-interruptible I/O may continue after its caller has timed out, consuming resources and possibly producing side effects.

Timeout on Future.get limits how long the caller waits; it does not automatically cancel the task. Decide whether to call cancel, whether work is safe to abandon, and whether downstream timeouts are also configured.

Shutdown Is a Lifecycle Protocol

`shutdown` stops new submissions and allows queued work to finish. `shutdownNow` attempts to interrupt running tasks and returns tasks that never started. A robust application waits for termination and escalates deliberately.

```
executor.shutdown();
if (!executor.awaitTermination(30, TimeUnit.SECONDS)) {
    List<Runnable> neverStarted = executor.shutdownNow();
    if (!executor.awaitTermination(10, TimeUnit.SECONDS)) {
        log.error("Executor did not terminate");
    }
}
```

Tasks must respond to interruption for shutdown to complete. Define what happens to accepted but unfinished business work during deployment: finish, persist for retry, or reject before shutdown.

Scheduled Execution

ScheduledExecutorService schedules delayed and periodic tasks. `scheduleAtFixedRate` targets a regular start cadence; `scheduleWithFixedDelay` waits a delay after one execution completes.

```
fixed rate:   start ---- start ---- start ---- target cadence
fixed delay:  run -- delay -- run -- delay -- run
```

A long fixed-rate task does not execute concurrently with itself through the same periodic schedule, but executions can bunch without sufficient delay. An exception escaping a periodic task suppresses later executions. Catch and report expected failures inside the task when continuation is required.

For durable, cluster-wide jobs, an in-process scheduler is insufficient by itself. Restarts lose schedules, and multiple instances may all run the job. Use persistent scheduling, leader election, or database-backed coordination according to requirements.

ThreadFactory and Operational Identity

A ThreadFactory can set meaningful names, daemon status, priority, and uncaught-exception handling.

```
ThreadFactory factory = runnable -> {
    Thread thread = new Thread(runnable);
    thread.setName("invoice-worker-" + sequence.incrementAndGet());
    thread.setUncaughtExceptionHandler((t, e) -> report(t, e));
    return thread;
};
```

Named threads make dumps and profiles readable. Avoid using daemon threads for work that must finish reliably; the JVM can exit when only daemon threads remain.

ThreadLocal Context Leakage

Pool threads outlive individual tasks. A ThreadLocal value left behind by one request can leak memory or contaminate the next request processed by that thread.

```
try {
    requestContext.set(context);
    task.run();
} finally {
    requestContext.remove();
}
```

Security identity, logging context, tracing, locale, and transaction context do not automatically propagate to arbitrary executors. Use framework-supported context propagation or explicit immutable context. Copying every ThreadLocal blindly can spread stale or unsafe state.

Pool Isolation and Bulkheads

One global pool allows a slow workload to consume every worker and delay unrelated work. Separate pools can isolate failure domains, but too many pools waste threads and hide total concurrency.

```
payment calls -> payment capacity boundary
report rendering -> bounded CPU executor
email delivery -> durable queue and workers
```

Partition by genuinely different resource or reliability requirements. Track total thread count, queue capacity, and downstream limits across the whole process.

Nested Submission Deadlock

A task in a fixed pool can submit child work to the same pool and block waiting for it. If every worker does this, no worker remains to execute the children.

```
pool size 2
worker 1 waits for child A queued to same pool
worker 2 waits for child B queued to same pool
no worker can run A or B
```

Avoid blocking dependency cycles within bounded executors. Compose without blocking, run child work directly when appropriate, use a separately sized executor with clear capacity, or redesign task ownership.

Production Observability

Monitor active workers, current and maximum pool size, queue length and oldest-task age, submission/completion/failure/rejection rates, task run time, queue wait time, cancellation, and shutdown progress. Queue age is often more informative than queue length.

Thread dumps distinguish RUNNABLE, WAITING, TIMED_WAITING, and BLOCKED states but require context. Correlate executor metrics with CPU, downstream pools, latency, and heap retention.

The Admission Algorithm Explains the Configuration

`ThreadPoolExecutor` does not simply fill the pool to `maximumPoolSize` and then queue. Its admission sequence is more specific:

```
submit task
    |
    +-- workers < corePoolSize? ------ yes --> start a worker with task
    |
    no
    +-- executor running and queue accepts? --> enqueue, then recheck state
    |
    no
    +-- workers < maximumPoolSize? --- yes --> start an extra worker
    |
    no
    +--------------------------------------> reject
```

This ordering creates a famous trap. With an effectively unbounded queue, almost every task after the core threads is queued, so the pool normally never grows toward `maximumPoolSize`. Core 10, maximum 200, and an unbounded `LinkedBlockingQueue` behaves under sustained load much more like a ten-thread executor with a growing waiting room than a 200-thread executor.

There is a race-aware recheck after enqueueing. The executor may have shut down between the first state check and queue insertion, or all workers may have disappeared. It removes and rejects the task after shutdown, or ensures that at least one worker exists. Admission and lifecycle are one protocol, not independent flags.

A Task Has More Than Running and Finished States

```
created -> submitted -> queued -> running -> completed normally
                    |          |       +--> completed exceptionally
                    |          +----------> cancellation requested
                    +---------------------> rejected
```

Cancellation before execution can prevent a queued task from running, although cancelled tasks may remain physically present in some queues until removal or maintenance. Cancellation during execution is a request, normally expressed through interruption. If task code ignores interruption or is stuck in a non-interruptible operation, the worker continues. A returned `Future` records completion state; it is not a forceful thread-termination mechanism.

Queue wait and execution time must be measured separately. If total latency is two seconds but execution takes 30 milliseconds, optimizing the task body cannot solve the incident. Record submission and start times so queue age becomes visible.

Capacity Planning with Arrival Rate and Service Time

Pool sizing formulas are hypotheses to test. For CPU-bound work, a pool near the number of processors often maximizes throughput. Extra runnable platform threads can add scheduling and cache costs without increasing completed work.

For blocking work, a common estimate is:

```
threads ~= processors * targetUtilization * (1 + waitTime / computeTime)
```

If a task computes for 10 ms and waits remotely for 90 ms, more threads may keep CPUs occupied. But downstream connection limits, memory per task, timeout budgets, and acceptable concurrency can impose lower ceilings. Increasing threads cannot make a ten-connection database pool serve 500 simultaneous queries; it merely relocates the queue.

Little's Law gives a durable model for a stable system:

```
in-flight work ~= arrival rate * time in system
```

At 1,000 tasks per second and an average 200 ms residence time, roughly 200 tasks are in flight. If only 40 execute, the rest wait somewhere. A bounded queue should represent waiting the service can tolerate, not the largest integer memory permits. Queue capacity, workers, and rejection together define the overload contract.

Rejection Policies Are Product Behavior

```
AbortPolicy          throw RejectedExecutionException; explicit failure
CallerRunsPolicy     submitting thread executes; feedback slows producer
DiscardPolicy        silently drop the new task
DiscardOldestPolicy  remove queue head and retry admission
custom handler       record, persist, route, or fail by domain contract
```

`CallerRunsPolicy` can provide backpressure when the caller is a normal producer: saturation consumes producer time and slows submission. It is dangerous when the caller is an event loop, holds a lock, or has a strict latency role. Running user code inline can block unrelated channels or unexpectedly extend a critical section.

Silent discard is acceptable only for explicitly best-effort work whose loss is measured. `DiscardOldestPolicy` is not inherently sensible either: the oldest item may be most important or closest to its deadline. A senior design states what callers observe, what can be lost, whether retry duplicates work, and which alert exposes rejection.

Observing Failures from execute and submit

With `execute(Runnable)`, an uncaught runtime exception normally escapes the task and can reach the worker thread's uncaught-exception handling; the worker may be replaced. With `submit`, work is wrapped in a `FutureTask`, which captures the exception as the future's result. If nobody calls `get` or otherwise records completion, a production failure can disappear.

Overriding `ThreadPoolExecutor.afterExecute` can centralize observation, but its `Throwable` argument is often null for submitted futures because the wrapper caught the exception. Robust instrumentation recognizes

a completed `Future`, calls `get`, and handles cancellation, execution failure, and interruption while preserving interrupt status. Completion callbacks or task wrappers are often clearer than relying on every caller.

A custom `ThreadFactory` should give stable names containing the pool and role, and deliberately configure daemon status and an uncaught exception handler where useful. Names such as `payment-db-7` make dumps and profiles more meaningful than `pool-12-thread-7`. The handler is an observation point, not a business recovery strategy.

Shutdown and Scheduling Details That Cause Incidents

Normal shutdown stops admission, allows queued work to finish, waits for a bounded period, and then requests interruption if abandonment is safe:

```
pool.shutdown();
try {
    if (!pool.awaitTermination(20, TimeUnit.SECONDS)) {
        List<Runnable> neverStarted = pool.shutdownNow();
        if (!pool.awaitTermination(10, TimeUnit.SECONDS)) {
            log.error("executor did not terminate");
        }
    }
} catch (InterruptedException e) {
    pool.shutdownNow();
    Thread.currentThread().interrupt();
}
```

`shutdownNow()` is best effort: it drains tasks that never commenced and interrupts workers, but cannot guarantee running code stops. The lifecycle must decide what happens to partial business operations and whether abandoned queued work requires durable recovery.

```
fixed rate target:   0----10----20----30----40
task lasts 7:        [=====]   [=====]   [=====]

fixed delay 10:      [=====]----------[=====]----------[=====]
                     completion + delay determines next start
```

Fixed-rate execution aims for the original cadence; if an invocation runs long, later starts may occur soon after completion, although executions of the same periodic task do not overlap. Fixed-delay execution waits after each completion. If a periodic task throws an exception that escapes, later executions are suppressed. Catch, record, and classify failures when continued scheduling is required.

Context Hygiene and Modern Executor Choices

Pooled platform threads are reused, so `ThreadLocal` state can leak identity, logging context, locale, or transaction-related data. Propagation should capture intended values at submission, install them before execution, and restore previous values in `finally`.

```
submitter context -> capture -> queued task -> install -> run -> restore
```

Restoration matters for nested execution: blindly clearing can destroy a legitimate outer context. A task decorator or context-aware executor keeps this protocol out of business code. Starting work on another thread normally leaves a caller's thread-bound transaction rather than extending it safely.

Java 21 virtual threads change the cost of blocking, not dependency capacity. A thread-per-task virtual-thread executor can scale straightforward blocking code beyond a platform pool, but it does not itself provide a bounded queue or concurrency ceiling. Use semaphores, connection pools, and rate limits around scarce resources. Retain a bounded platform pool when CPU concurrency itself needs limiting or a library is unsuitable for large virtual-thread concurrency.

A Strong Interview Explanation

An executor separates task submission from execution policy. For `ThreadPoolExecutor`, explain core workers, queueing, extra workers to maximum, and finally rejection, because the queue determines whether maximum size is ever used. Treat worker count, queue bound, and rejection as one overload contract.

Size CPU work near processor capacity and blocking work from measured wait/compute ratios while respecting downstream limits. Observe queue wait separately from run time, capture exceptions from futures, preserve interruption, propagate context deliberately, and shut down with a bounded lifecycle. Isolate genuine failure domains without creating a pool for every method. Virtual threads simplify blocking concurrency, but scarce dependencies still need explicit bulkheads.

Interview-Ready Summary

Executors separate tasks from execution policy. ThreadPoolExecutor behavior emerges from worker counts, queue type and capacity, thread creation, rejection, and lifecycle rules. An unbounded queue hides overload as retained latency and memory.

Size CPU and waiting workloads from measurement and downstream capacity. Observe submitted task failures, preserve interruption, and shut down explicitly. Use named threads, clean ThreadLocal context, isolate genuine failure domains, and avoid tasks that block waiting for children in the same saturated pool.

Revision Notes

- Runnable and Callable describe work; Executor defines execution policy.
- Submission does not guarantee immediate execution.
- Core size, maximum size, queue, and rejection must be designed together.
- An unbounded queue normally prevents growth beyond core threads.
- Queue length represents retained memory and waiting latency.
- Bounded queues force an explicit overload policy.
- CPU-bound concurrency is limited by available processing capacity.
- Waiting workloads remain limited by downstream resources.
- Rejection behavior is part of the service contract.
- Discard policies can silently lose business work.
- submit captures failures in Future.
- Ignored futures can hide task exceptions.
- Cancellation is cooperative interruption, not forced termination.
- A timed get does not automatically stop the task.
- shutdown and shutdownNow have different lifecycle semantics.
- Periodic task exceptions can suppress future executions.
- In-process scheduling is not automatically durable or cluster-safe.
- Thread names materially improve production diagnosis.
- Pool ThreadLocal values must be cleaned and propagated deliberately.
- Avoid blocking task dependency cycles inside the same bounded pool.
- Observe queue age, task latency, failures, rejection, and downstream capacity.
- Admission order is core workers, queue, extra workers, then rejection.
- An unbounded queue commonly makes maximumPoolSize ineffective.
- Measure queue wait separately from task execution time.
- Little's Law connects arrival rate, residence time, and in-flight work.
- Queue capacity represents tolerated waiting, not available heap.
- CallerRunsPolicy applies feedback but may block a critical caller.
- shutdownNow requests interruption; it cannot force code to stop.
- Fixed rate follows target cadence; fixed delay starts after completion plus delay.
- Restore prior thread context after pooled execution.
- Virtual threads reduce blocking cost but do not bound scarce resources.